DevOps Infrastructure – Unit 2 Notes (Part 4)

Docker Storage: Volumes, Bind Mounts and Data Persistence

---

1. Docker Storage – Overview

Problem

Containers are ephemeral in nature:

- When a container is deleted, its data is lost

Solution

Docker provides storage mechanisms to persist data:

- Volumes
- Bind mounts

---

2. Docker Volumes

Definition

Volumes are managed storage locations created and maintained by Docker.

Key Features

- Data persists beyond container lifecycle
- Stored in Docker-managed directories
- Can be shared between multiple containers
- More secure and portable

Creating a Volume

```
docker volume create my_volume
```

Using a Volume

```
docker run -v my_volume:/app/data my_app
```

Advantages

- Easy to manage
- Better performance
- Works across different environments

---

3. Bind Mounts

Definition

Bind mounts link a directory from the host system directly into a container.

Example

docker run -v /host/data:/app/data my_app

Key Features

- Direct access to host files
- Real-time synchronization
- Useful for development

Advantages

- Easy debugging
- Instant file updates

Disadvantages

- Less secure
- Dependent on host directory structure

---

4. Volumes vs Bind Mounts

Feature	Volumes	Bind Mounts
Managed by Docker	Yes	No
Portability	High	Low
Performance	Better	Slightly lower
Use Case	Production	Development

---

5. Backing Data on Host

Concept

Even though volumes are managed by Docker, they are physically stored on the host system.

Default Location (Linux)

Typically under:
/var/lib/docker/volumes/

Importance

- Data persists even if container is removed
- Can be backed up or migrated

---

6. Summary

- Containers lose data when removed
- Volumes provide persistent, Docker-managed storage
- Bind mounts connect host directories directly
- Volumes are preferred for production systems